



Orders Management - Assignment 3

Fundamental Programming Techniques, Computer Science Department, TUCN

Spring 2026

1 Requirements

Goal: Design and implement an application for orders management with the objective of processing client orders for a warehouse.

Consider an application **Orders Management** for processing client orders for a warehouse. Relational databases should be used to store the products, the clients and the orders. The application should be designed according to the layered architecture pattern and should use (minimally the following classes):

1. **Model classes:** represent the data models of the application
2. **Business Logic classes:** contain the application logic
3. **Presentation classes:** GUI related classes
4. **Data access classes:** classes that contain the access to the database

Note: Other classes and packages can be added to implement the full functionality of the application.

2 Deliverables

1. **Source code files, JavaDoc files, SQL dump file(containing the SQL statements for creating the database and the tables, and populating the tables with the corresponding data):** will be uploaded on the personal **Gitlab** account created according to the instructions in the **Laboratory Resources** document, and the following steps:
 - (a) Create a private repository on **Gitlab** named according to the following template
PT2026_Group_LastName_FirstName_Assignment_3
 - (b) Push the source code (**not as archive**).
 - (c) Share the repository with the user **utcn_dsrl** with Maintainer as role.
2. The **draw.io** file with the UML use case diagram, package diagram and the class diagram will be uploaded in the repository with the source code files.

3 Evaluation

The assignment will be graded as follows:



Requirements	Grading
1. Use an object-oriented programming design, classes with maximum 300 lines (except UI classes) and methods with maximum 30 lines, Java naming conventions. 2. Use <i>javadoc</i> for documenting classes and generate the corresponding JavaDoc files. 3. Use relational databases for storing the data for the application, minimum three tables: Client, Product and Order. 4. Create a graphical user interface including: (a) A window for client operations: add a new client, edit client, delete client, view all clients in a table (JTable) (b) A window for product operations: add a new product, edit product, delete product, view all products in a table (JTable) (c) A window for creating product orders - the user will be able to select an existing product, select an existing client, and insert a desired quantity for the product to create a valid order. In case there are not enough products, an under-stock message will be displayed. After the order is finalized, the product stock is decremented. (d) Use reflection techniques to create a method that receives a list of objects and generates the header of the table by extracting through reflection the object properties and then populates the table with the values of the elements from the list.	5 points
Implement a layered architecture (the application will contain at least 4 packages : dataAccessLayer , businessLayer , model and presentation)	+1 point
Define an immutable Bill class in the Model package using Java records. A Bill object will be generated for each order and will be stored in a Log table. The bills can only be inserted and read from the Log table; no updates are allowed.	+1 point
Use reflection techniques to create a generic class that contains the methods (create, edit, delete, select/find) for accessing the DB where applicable.	+2 point
Use lambda expressions and streams for list and array processing	+1 point

4 Bibliography

1. Connect to MySql from a Java application: <https://www.baeldung.com/java-jdbc>
2. Layered architectures : <https://dzone.com/articles/layers-standard-enterprise>
3. Reflection in Java: <https://jenkov.com/tutorials/java-reflection/index.html>
4. JAVADOC: <https://www.baeldung.com/javadoc>
5. SQL dump file generation: <https://dev.mysql.com/doc/workbench/en/wb-admin-export-import-management.html>